

```

doubleTree(struct node* node) {
    struct node* oldLeft;

    node==NULL return;

    // traverse the subtrees
    doubleTree(node->left);
    doubleTree(node->right);

    // duplicate this node to its left
    node->left = node->left;
    node->->left = newNode(node->data);
    node->->->left = oldLeft;

    // sameTree
    int sameTree(struct node* a, struct node* b) {
        // 1. both empty -> true
        if (a==NULL && b==NULL) return(true);
        // 2. both non-empty -> compare them
        else if (a!=NULL && b!=NULL) {
            return(
                a->data == b->data &&
                sameTree(a->left, b->left) &&
                sameTree(a->right, b->right)
            );
        }
        // 3. one empty, one not -> false
        else return(false);
    }

    // Returns true if a binary tree
    // is a binary search tree.
    int isBST(struct node* node) {
        if (node==NULL) return(true);
        if (node->left!=NULL && maxVal(node->left) >
            node->data)
            return(false);
        if (node->right!=NULL && minVal(node->right)
            <= node->data)
            return(false);
        if (!isBST(node->left) || !isBST(node->right))
            return(false);
        return(true);
    }

    // MIN AND MAX
    void min_max(N *temp)
    {
        while (temp->l != NULL)
            temp = temp->l;
        printf("Minimum value = %d\n", temp->value);
        temp = root;
        while (temp->r != NULL)
            temp = temp->r;
        printf("Maximum value = %d\n", temp->value);
    }

    // create a balanced binary search tree
    N* bst(int arr[], int first, int last)
    {
        int mid;
        N* temp = (N*)malloc(sizeof(N));
        if (first > last)
            return NULL;
        mid = (first + last) / 2;
        temp = new(arr[mid]);
        temp->l = bst(arr, first, mid - 1);
        temp->r = bst(arr, mid + 1, last);
        return temp;
    }

    // Find the Sum of all Nodes in a Tree
    int modify_tree(struct btnode *tree)
    {
        int i, original;

        if (tree == NULL)
            return 0;
        original = tree->value;
        tree->value = modify_tree(tree->l)
        + modify_tree(tree->r);
        return tree->value + original;
    }
}

2 5 7 8 | 1 -- 1 2 5 7 8 | 3
void insertionSort(int D[], int n)
{
    int i, k, key;
    for (i = 1; i < n; i++)
        key = D[i];

    for (k = i - 1; k >= 0 && key <= D[k]; k--)
        D[k+1] = D[k]; // shift operation
    D[k+1] = key; // insert key
}

2 5 7 8 3 1 -- 1 5 7 8 3 2 -- 1 5 7 8
void selectionSort(int D[], int n) {
    int i, index, j, min;
    for (i = 0; i < (n - 1); i++) {
        min = D[n - 1];
        index = n - 1;
        for (j = i; j < (n - 1); j++) {
            if (D[j] < min) {
                min = D[j];
                index = j;
            }
        }
        if (i != index) {
            D[index] = D[i];
            D[i] = min;
        }
    }
}

2 5 7 8 3 1 -- 2 5 3 7 8 1
void bubbleSort(int D[], int n)
{
    int temp, k, move;

    for (move = 0; move < (n - 1); move++) {
        for (k = 0; k < (n - 1 - move); k++) {
            if (D[k] > D[k + 1]) //exchange the values
            {
                temp = D[k];
                D[k] = D[k + 1];
                D[k + 1] = temp;
            }
        }
    }
}

// merge sort ikiye bölünerek devam eder 5->3-2
// quick sort pivot mantığı ile çözülürken left - p - right
// heap sort tree mantığı ile yazılacak değiştirilecek

```